

Konstrukturen und Garbage Collection – Teil 2

`super(...)`, `this(...)` und GC

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 8. Januar 2026 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Konstrukturen und Garbage Collection

 Plan für diese Woche

 Stack und Heap

 Konstrukturen

 Objektinitialisierung

 Überladene Konstrukturen I

 Superklassenkonstrukturen

 Überladene
Konstrukturen II

 Pair Programming

 Garbage Collection

 Spaß

📖 Konstruktoren und Garbage Collection

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

UP! Welche Rolle spielen Superklassenkonstruktoren? Wie hängen sie mit dem Vererbungsbaum zusammen?

🧱 Wie können Konstruktoren, die zur selben Klasse gehören, miteinander interagieren?

♻️ Wie lange „lebt“ eine Variable? Wie unterscheidet sich die Lebensdauer zwischen lokalen Variablen und Instanzvariablen?

🙋 Welche Fragen haben Sie zu diesem Kapitel?

9 Konstruktoren und Garbage Collection

Leben und Sterben eines Objekts



Objekte werden geboren und Objekte sterben. Sie als Programmierer/-in sind für den Lebenszyklus eines Objekts verantwortlich. Sie entscheiden, wie es **konstruiert** wird. Sie entscheiden, wenn es **zerstört** wird. Nur, dass Sie das Objekt nicht selbst *zerstören*, Sie *geben* es einfach *auf*. Aber sobald Sie es aufgegeben haben, kann der herzlose **Garbage Collector (gc)** es pulverisieren und den Speicher, den das Objekt belegte, wieder anderweitig nutzen. Wenn Sie Java schreiben, werden Sie Objekte erstellen. Früher oder später müssen Sie einige von ihnen gehen lassen, oder Sie riskieren, dass der Arbeitsspeicher knapp wird. In diesem Kapitel sehen wir uns an, wie Objekte erstellt werden, wo sie leben, während sie leben, und wie man sie effizient gehen lässt. Das heißt, wir sprechen über den Heap, den Stack, den Geltungsbereich (Scope), Konstruktoren, Superklassenkonstruktoren, Nullreferenzen und mehr. Warnung: Dieses Kapitel enthält Material über den Tod von Objekten, den Sie als Lesende möglicherweise beunruhigend finden. Am besten gehen Sie keine emotionale Bindung ein.

Superklassenkonstruktoren



**Jetzt Seiten 252
und 253
wiederholen!**



Sie haben dafür 5 Minuten Zeit.

Ein Stack-Szenario

Superklassenkonstruktoren

```
public class Animal {  
    public Animal() {  
        System.out.println("Making an Animal");  
    }  
}
```

```
public class Hippo extends Animal {  
    public Hippo() {  
        System.out.println("Making a Hippo");  
    }  
}
```

```
public class TestHippo {  
    public static void main(String[] args) {  
        System.out.println("Starting...");  
        Hippo h = new Hippo();  
    }  
}
```



Wie lautet die richtige Ausgabe?
Finden Sie anhand des Codes auf der linken Seite heraus, welche Ausgaben bei der Ausführung von TestHippo erzeugt werden. A oder B?

(Die Antwort finden Sie unten auf dieser Seite.)

A

Datei Bearbeiten Fenster Hilfe Fluchen

```
% java TestHippo  
Starting...  
Making an Animal  
Making a Hippo
```

B

Datei Bearbeiten Fenster Hilfe Fluchen

```
% java TestHippo  
Starting...  
Making a Hippo  
Making an Animal
```





Ein Stack-Szenario

UP! Superklassenkonstruktoren

```
public class Animal {  
    public Animal() {  
        System.out.println("Making an Animal");  
    }  
}
```

```
public class Hippo extends Animal {  
    public Hippo() {  
        System.out.println("Making a Hippo");  
    }  
}
```

```
public class TestHippo {  
    public static void main(String[] args) {  
        System.out.println("Starting...");  
        Hippo h = new Hippo();  
    }  
}
```



Spitzen Sie Ihren Bleistift

Wie lautet die richtige Ausgabe?
Finden Sie anhand des Codes auf der linken Seite heraus, welche Ausgaben bei der Ausführung von TestHippo erzeugt werden. A oder B?

(Die Antwort finden Sie unten auf dieser Seite.)

A

Datei Bearbeiten Fenster Hilfe Fluchen

```
% java TestHippo  
Starting...  
Making an Animal  
Making a Hippo
```

~~B~~

Datei Bearbeiten Fenster Hilfe Fluchen

```
% java TestHippo  
Starting...  
Making a Hippo  
Making an Animal
```

💡 Ein Stack-Szenario

UP! Superklassenkonstruktoren

```
public class Animal {  
    public Animal() {  
        System.out.println("Making an Animal");  
    }  
}
```

```
public class Hippo extends Animal {  
    public Hippo() {  
        System.out.println("Making a Hippo");  
    }  
}
```

```
public class TestHippo {  
    public static void main(String[] args) {  
        System.out.println("Starting...");  
        Hippo h = new Hippo();  
    }  
}
```

① Der Code einer anderen Klasse ruft **new Hippo()** auf, und der **Hippo()**-Konstruktor wird in den Stack-Frame an der Spitze des Stacks eingefügt.

Hippo()

② **Hippo()** ruft den Superklassenkonstruktor auf, wodurch der **Animal()**-Konstruktor an die Spitze des Stacks geschoben wird.

Animal()
Hippo()

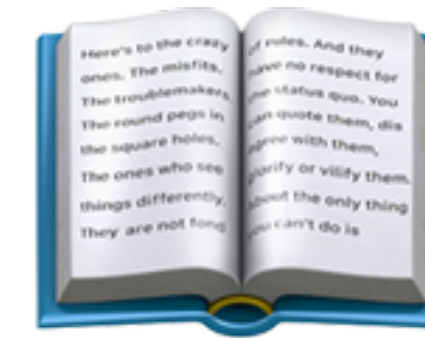
③ **Animal()** ruft den Superklassenkonstruktor auf, was den **Object()**-Konstruktor an die Spitze des Stacks setzt, weil Object die Superklasse von Animal ist.

Object()
Animal()
Hippo()

④ **Object()** wird beendet, und sein Stack-Frame wird vom Stack entfernt. Die Ausführung kehrt zum **Animal()**-Konstruktor zurück und macht auf der Zeile weiter, die auf den Aufruf von **Animal()** an seinen Superklassenkonstruktor folgt.

Animal()
Hippo()

**Jetzt Seiten 255
und 256
wiederholen!**



Sie haben dafür 5 Minuten Zeit.


```
1 public class SuperDemo {  
2  
3     public static void main(String[] args) {  
4         Dog rex = new Dog("Rex", "German shepherd");  
5         System.out.println(rex);  
6  
7         Dog daisy = new Dog("Daisy");  
8         System.out.println(daisy);  
9  
10        Ant ant = new Ant();  
11        System.out.println(ant);  
12  
13        // Ant antMan = new Ant("Ant-Man");  
14  
15    }  
16  
17 }  
18
```

STDIN

Input for the program (Optional)

Output:

```
Dog[German shepherd] is an Animal[Rex]  
Dog[mongrel] is an Animal[Daisy]  
Ant
```




Spielen Sie Compiler!

UP! Superklassenkonstruktoren

 Seite
259

```
public class Boo {  
    public Boo(int i) { }  
    public Boo(String s) { }  
    public Boo(String s, int i) { }  
}
```

```
class SonOfBoo extends Boo {  
    public SonOfBoo() {  
        super("boo");  
    }  
  
    public SonOfBoo(int i) {  
        super("Fred");  
    }  
  
    public SonOfBoo(String s) {  
        super(42);  
    }  
  
    public SonOfBoo(int i, String s) {  
    }  
  
    public SonOfBoo(String a, String b, String c) {  
        super(a, b);  
    }  
  
    public SonOfBoo(int i, int j) {  
        super("man", j);  
    }  
  
    public SonOfBoo(int i, int x, int y) {  
        super(i, "star");  
    }  
}
```

Einige Konstruktoren der Klasse SonOfBoo können nicht kompiliert werden. Finden Sie heraus, welche Konstruktoren **ungültig** sind.

Ordnen Sie dazu den ungültigen Konstruktoren die entsprechenden **Fehlermeldungen des Compilers** zu.



Spielen Sie Compiler!



Superklassenkonstruktoren

```
public class Boo {  
    public Boo(int i) { }  
    public Boo(String s) { }  
    public Boo(String s, int i) { }  
}
```

```
class SonOfBoo extends Boo {  
    public SonOfBoo() {  
        super("boo");  
    }  
  
    public SonOfBoo(int i) {  
        super("Fred");  
    }  
  
    public SonOfBoo(String s) {  
        super(42);  
    }  
  
    public SonOfBoo(int i, String s) {  
    }  
  
    public SonOfBoo(String a, String b, String c) {  
        super(a, b);  
    }  
  
    public SonOfBoo(int i, int j) {  
        super("man", j);  
    }  
  
    public SonOfBoo(int i, int x, int y) {  
        super(i, "star");  
    }  
}
```

```
Datei Bearbeiten Fenster Hilfe Rumpel  
%javac SonOfBoo.java  
cannot resolve symbol  
symbol : constructor Boo  
(java.lang.String,java.  
lang.String)
```

```
Datei Bearbeiten Fenster Hilfe Pumpel  
%javac SonOfBoo.java  
cannot resolve symbol  
symbol : constructor Boo  
(int,java.lang.String)
```

```
Datei Bearbeiten Fenster Hilfe KrachStraße  
%javac SonOfBoo.java  
cannot resolve symbol  
symbol:constructor Boo()
```

! ? Quiz

UP! Superklassenkonstruktoren

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Es gibt keine andere Möglichkeit, ein Objekt zu erzeugen, als `new` aufzurufen.

Richtig!

Abstrakte Klassen können keine
Konstruktoren enthalten, da sie abstrakt
sind und nicht instanziiert werden können.

Ihr Konstruktor wird ausgeführt, sobald jemand eine Instanz einer konkreten Unterklasse erzeugt.

Zur Erzeugung eines neuen Objekts müssen alle Konstruktoren im Vererbungsbaum des Objekts durchlaufen werden.

Richtig!

Der Superklassenkonstruktor wird durch den Namen der Superklasse aufgerufen, z. B.

```
public Duck() { Animal(); }
```

`super()` ruft den Superklassenkonstruktor auf.

Wenn Sie keinen Konstruktor deklarieren, fügt der Compiler einen für Sie ein, so wie es hier der Fall ist:

```
public ClassName() {  
    super();  
}
```

Richtig!

Wenn ein Konstruktor ausgeführt wird, ruft er sofort seinen Superklassenkonstruktor auf, und zwar die ganze Kette bis zum Konstruktor von `Object`.

Zumindest dann, wenn der Konstruktor keinen anderen überladenen Konstruktor aufruft.

 Welcher **eine** Gedanke zu
„ Superklassen-
konstruktoren“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren



Überladene Konstruktoren II

Jetzt Seite 258 wiederholen!



Sie haben dafür 3 Minuten Zeit.

```
1 public class OverloadingDemo {  
2  
3     public static void main(String[] args) {  
4         Dog rex = new Dog("Rex", "German shepherd");  
5         System.out.println(rex);  
6  
7         Dog daisy = new Dog("Daisy");  
8         System.out.println(daisy);  
9     }  
10  
11 }  
12
```

STDIN

Input for the program (Optional)

Output:

Dog[German shepherd] is an Animal[Rex]
Dog[mongrel] is an Animal[Daisy]

! ? Quiz

📦 Überladene Konstruktoren II

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Wenn Sie dies nicht selbst tun, fügt der Compiler einen Aufruf von `super()` in Ihre Konstruktoren ein. Zumindest dann, wenn Ihre Konstruktoren keinen anderen überladenen Konstruktor aufrufen.

Richtig!

Wenn Sie einen Konstruktor bereitstellen, aber keinen Aufruf von `super(...)` einfügen, fügt der Compiler einen Aufruf von `this()` in jeden Ihrer überladenen Konstruktoren ein. Zumindest solange Ihre Konstruktoren keinen anderen überladenen Konstruktor aufrufen.

... einen Aufruf von `super()`...

Der Aufruf von `super(...)` muss die erste Anweisung in jedem Konstruktor sein! Dies gilt zumindest so lange, wie Ihr Konstruktor keinen anderen überladenen Konstruktor aufruft.

Richtig! Unberücksichtigt Java Enhancement Proposal (JEP) 447.

Um einen Konstruktor aus einem anderen überladenen Konstruktor derselben Klasse aufzurufen, verwenden Sie `here(...)`.

... verwenden Sie `this(...)`.

Der Aufruf von `this(...)` darf nur innerhalb eines Konstruktors verwendet werden und muss dann die erste Anweisung innerhalb des Konstruktors sein.

Richtig! Unberücksichtigt Java Enhancement Proposal (JEP) 447.

Ein Konstruktor kann einen Aufruf von `super(...)` oder einen Aufruf von `this(...)` enthalten – oder beide.

... aber niemals beide!

 Welcher **eine** Gedanke zu
„ Überladene
Konstruktoren II“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 Konstrukturen und Vererbung

🤝 Pair Programming

- 🎯 Erstellen Sie eine Vererbungshierarchie mit Klassen, die Konstruktoren definieren und aufrufen.
- ✅ Erstellen Sie eine Klassenhierarchie mit einer Oberklasse Person und zwei Unterklassen Student und Professor.
- ✅ Implementieren Sie in jeder Klasse Konstruktoren unter Verwendung der Schlüsselwörter `this` und `super`, um die Objekte geeignet zu initialisieren.
- ✅ Jede Person hat die grundlegenden Attribute `name` und `age`. Der Name muss immer vorhanden sein, das Alter ist optional.



Konstrukturen und Vererbung

Pair Programming

 **Erstellen Sie eine Vererbungshierarchie mit Klassen, die Konstruktoren definieren und aufrufen.**

 Erstellen Sie eine Klassenhierarchie mit einer Oberklasse Person und zwei Unterklassen Student und Professor.

 Implementieren Sie in jeder Klasse Konstruktoren unter Verwendung der Schlüsselwörter `this` und `super`, um die Objekte geeignet zu initialisieren.

 Jede Person hat die grundlegenden Attribute `name` und `age`. Der Name muss immer vorhanden sein, das Alter ist optional.

 Implementieren Sie in jeder Klasse eine Methode, um die Details der Person anzuzeigen, z. B. `displayDetails()`.

 Studenten haben eine Matrikelnummer (`studentId`).

 Professoren haben eine spezifische Bezeichnung der Professur, die Denomination (`denomination`). Professoren haben kein Alter (d. h. ihr Alter wird nicht gespeichert).

 Implementieren Sie eine Klasse, die beispielhaft eine `Student:in` und eine `Professor:in` ausgibt.

 Auf der Konsole werden die Eigenschaften der `Student:in` und der `Professor:in` angezeigt.

```
1 public class SuperclassDemo {  
2  
3     public static void main(String[] args) {  
4         Student louis = new Student("Emma Dupont", 22, 15470811);  
5         louis.displayDetails();  
6  
7         System.out.println();  
8  
9         Professor martin =  
10            new Professor("Martin Burger", "Software Engineering und Software Quality Assurance");  
11         martin.displayDetails();  
12     }  
13  
14 }  
15
```

STDIN

Input for the program (Optional)

Output:

Person Details:

- Name: Emma Dupont
- Age: 22
- Student Details:
 - Student ID: 15470811

Person Details:

- Name: Martin Burger
- Age: n/s
- Professor Details:
 - Denomination: Software Engineering und Software Quality Assurance

 Welcher **eine** Gedanke zu
„ Konstruktoren und
Vererbung“ war für Sie am
wichtigsten?  Schreiben Sie ihn
auf!

 Durchatmen und  Sortieren



Garbage Collection



Lebensdauer von Objekten

Garbage Collection

- Ein **Objekt** entsteht durch den new-Operator.
- Das Leben eines Objekts hängt vollständig vom Leben der **Referenzen** ab, die auf es verweisen.
- Solange die Referenz als „lebendig“ gilt, lebt auch das Objekt auf dem Heap.
- Wenn die Referenz stirbt (was das bedeutet, werden wir gleich sehen), stirbt auch das Objekt.



Lebensdauer von Variablen

Garbage Collection

- Eine **Variable** entsteht, wenn sie deklariert wird.
- Ihre Lebensdauer hängt davon ab, ob es sich um eine **lokale** Variable oder eine **Instanzvariable** handelt.
- Für die Lebensdauer spielt es keine Rolle, ob es sich um eine elementare Variable oder um eine Referenzvariable handelt.



Lebensdauer von Variablen

Garbage Collection

- ① Eine lokale Variable lebt nur innerhalb der Methode, in der sie deklariert wurde.

```
public void read() {  
    int s = 42;  
    // 's' kann nur innerhalb  
    // dieser Methode benutzt  
    // werden.  
    // Endet die Methode,  
    // verschwindet 's'  
    // vollständig.  
}
```

Die Variable `s` kann nur innerhalb der Methode `read()` verwendet werden. Der Geltungsbereich einer Variablen ist auf ihre eigene Methode beschränkt.

Der Geltungsbereich der Variablen `s` (hier ein Parameter) ist auf die Methode `setSize()` beschränkt. Die Instanzvariable `size` ist während der gesamten Lebensdauer des Objekts gültig.

- ② Alle Instanzvariablen leben so lange wie ihr Objekt. Ist das Objekt noch am Leben, sind es auch seine Instanzvariablen.

```
public class Life {  
    int size;  
  
    public void setSize(int s) {  
        size = s;  
        // 's' verschwindet am Ende  
        // dieser Methode, aber  
        // 'size' kann überall in  
        // der Klasse benutzt werden.  
    }  
}
```

Leben und Geltungsbereich lokaler Variablen

Garbage Collection

Leben

- Eine lokale Variable lebt, solange sich ihr Stack-Frame auf dem Stack befindet, d. h. bis die Methode endet.

Geltungsbereich

- Der Geltungsbereich einer lokalen Variablen ist auf die Methode beschränkt, in der sie deklariert wurde.

- Wenn eine Methode eine andere aufruft, ist die Variable zwar lebendig, befindet sich aber außerhalb des aktuellen Geltungsbereichs, bis die aufrufende Methode fortgesetzt wird.
- Eine Variable kann nur innerhalb ihres Geltungsbereichs verwendet werden.

Ein Stack-Szenario

Garbage Collection

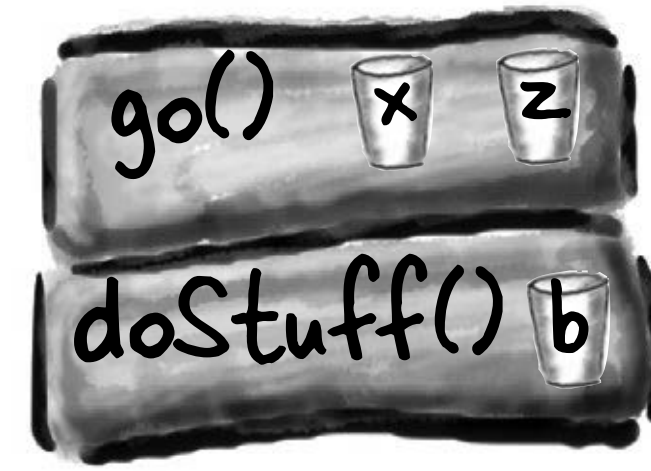
1 `public void doStuff() {
 boolean b = true;
 go(4);
}`

2 `public void go(int x) {
 int z = x + 24;
 crazy();
 // mehr Code hier
}`

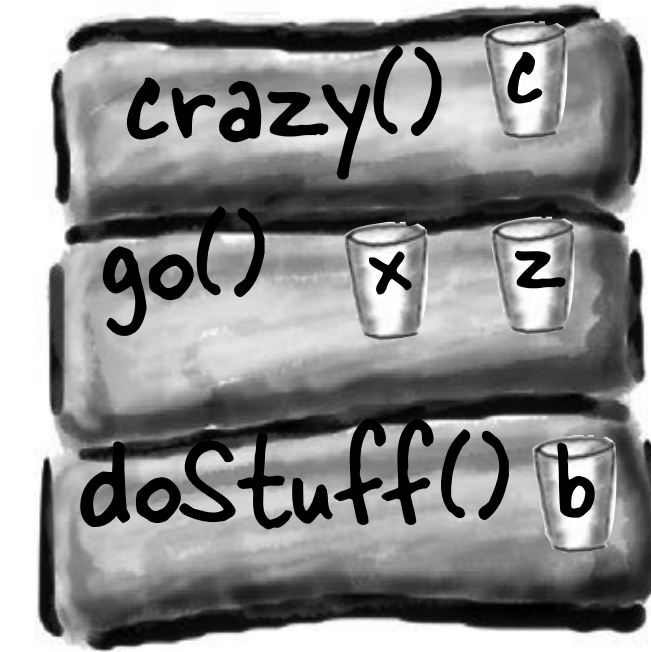
3 `public void crazy() {
 char c = 'a';
}`



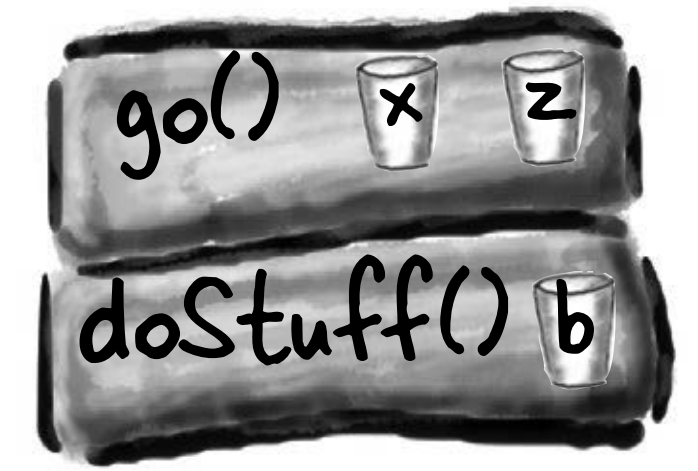
1 *doStuff()* landet auf dem Stack. Die Variable »b« ist lebendig und befindet sich im aktuellen Geltungsbereich.



2 *go()* rutscht an die Spitze des Stacks. »x« und »z« sind lebendig und im aktuellen Geltungsbereich, »b« ist lebendig, aber *nicht* im aktuellen Geltungsbereich.



3 *crazy()* wird auf den Stack geschoben. Jetzt ist »c« lebendig und im Geltungsbereich. Die anderen drei Variablen sind lebendig, befinden sich aber außerhalb des aktuellen Geltungsbereichs.



4 *crazy()* wird beendet und verschwindet vom Stack. »c« befindet sich also außerhalb des aktuellen Geltungsbereichs und *ist tot*. Wenn *go()* weitermacht, wo es pausiert hat, sind »x« und »z« beide lebendig und befinden sich wieder im aktuellen Geltungsbereich. Die Variable »b« ist ebenfalls noch lebendig, aber nicht mehr im aktuellen Geltungsbereich (bis *go()* beendet wird).

**Jetzt Seiten 262
bis 265
wiederholen!**



Sie haben dafür 9 Minuten Zeit.



Seien Sie der Garbage Collector!

Garbage Collection

 Seite
268

```
public class GC {
    public static GC doStuff() {
        GC newGC = new GC();
        doStuff2(newGC);
        return newGC;
    }

    public static void main(String[] args) {
        GC gc1;
        GC gc2 = new GC();
        GC gc3 = new GC();
        GC gc4 = gc3;
        gc1 = doStuff();

        A

        // weitere Methoden aufrufen
    }

    public static void doStuff2(GC copyGC) {
        GC localGC = copyGC;
    }
}
```

```
1  copyGC = null;
2  gc2 = null;
3  newGC = gc3;
4  gc1 = null;
5  newGC = null;
6  gc4 = null;
7  gc3 = gc2;
8  gc1 = gc4;
9  gc3 = null;
```



Welche der **Codezeilen** in der Mitte würde, wenn sie an Punkt A eingefügt würde, genau ein zusätzliches Objekt für die Garbage Collection freigeben?

Nehmen Sie an, dass Punkt A // weitere Methoden aufrufen sehr lange ausgeführt wird, damit die GC Zeit hat, ihren Job zu tun.



Seien Sie der Garbage Collector!



Garbage Collection

```
public class GC {  
    public static GC doStuff() {  
        GC newGC = new GC();  
        doStuff2(newGC);  
        return newGC;  
    }  
  
    public static void main(String[] args) {  
        GC gc1;  
        GC gc2 = new GC();  
        GC gc3 = new GC();  
        GC gc4 = gc3;  
        gc1 = doStuff();  
  
        A  
  
        // weitere Methoden aufrufen  
    }  
  
    public static void doStuff2(GC copyGC) {  
        GC localGC = copyGC;  
    }  
}
```

1 copyGC = null;

2 gc2 = null;

3 newGC = gc3;

4 gc1 = null;

5 newGC = null;

6 gc4 = null;

7 gc3 = gc2;

8 gc1 = gc4;

9 gc3 = null;

! ? Quiz

♻️ Garbage Collection

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Eine lokale Variable „lebt“ nur innerhalb der Methode, in der sie deklariert wurde. Der Geltungsbereich (Scope) einer solchen Variablen ist also auf ihre eigene Methode beschränkt.

Richtig!

Eine Variable kann nur innerhalb ihres Geltungsbereichs verwendet werden. Außerhalb dieses Scopes ist die Variable undefiniert und führt beim Zugriff zu einem Kompilierungsfehler.

Richtig!

Alle Instanzvariablen leben so lange wie ihr Objekt. Ist das Objekt noch am Leben, so sind es auch seine Instanzvariablen.

Richtig!

Die Garbage Collection (GC) sorgt dafür, dass der von nicht mehr „lebenden“ Objekten belegte Arbeitsspeicher (RAM) wieder freigegeben wird. Sie gibt den belegten Speicher an den Heap zurück, damit er für neue Objekte verwendet werden kann.

Richtig!

Ein Objekt qualifiziert sich für die Garbage Collection, wenn seine letzte „lebende“ Referenz verschwindet. Solche Objekte gelten als „garbage“ (Müll) und können von der GC eingesammelt werden, um den von ihnen belegten Speicher wieder freizugeben.

Richtig!

Eine Objektreferenz verliert ihre Gültigkeit, wenn die Referenz den Geltungsbereich endgültig verlässt. Zum Beispiel in Form einer lokalen Variable am Ende der Methode, in der die Referenz deklariert wurde.

Richtig!

Eine Objektreferenz wird ungültig, wenn Sie ihr explizit eine Instanz von `Object` zuweisen.

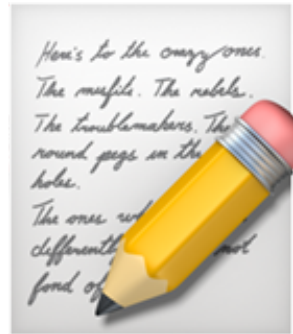
Sie explizit auf `null` setzen.

Wenn Sie den Punktoperator (.) auf einer Nullreferenz verwenden, versucht das Programm auf ein Objekt zuzugreifen, das nicht existiert (weil die Referenz `null` ist). Dies führt zur Laufzeit zu einer `NullPointerException`.

Richtig!



Welcher **eine** Gedanke zu
„ Garbage Collection“ war
für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Konstruktoren und Garbage Collection

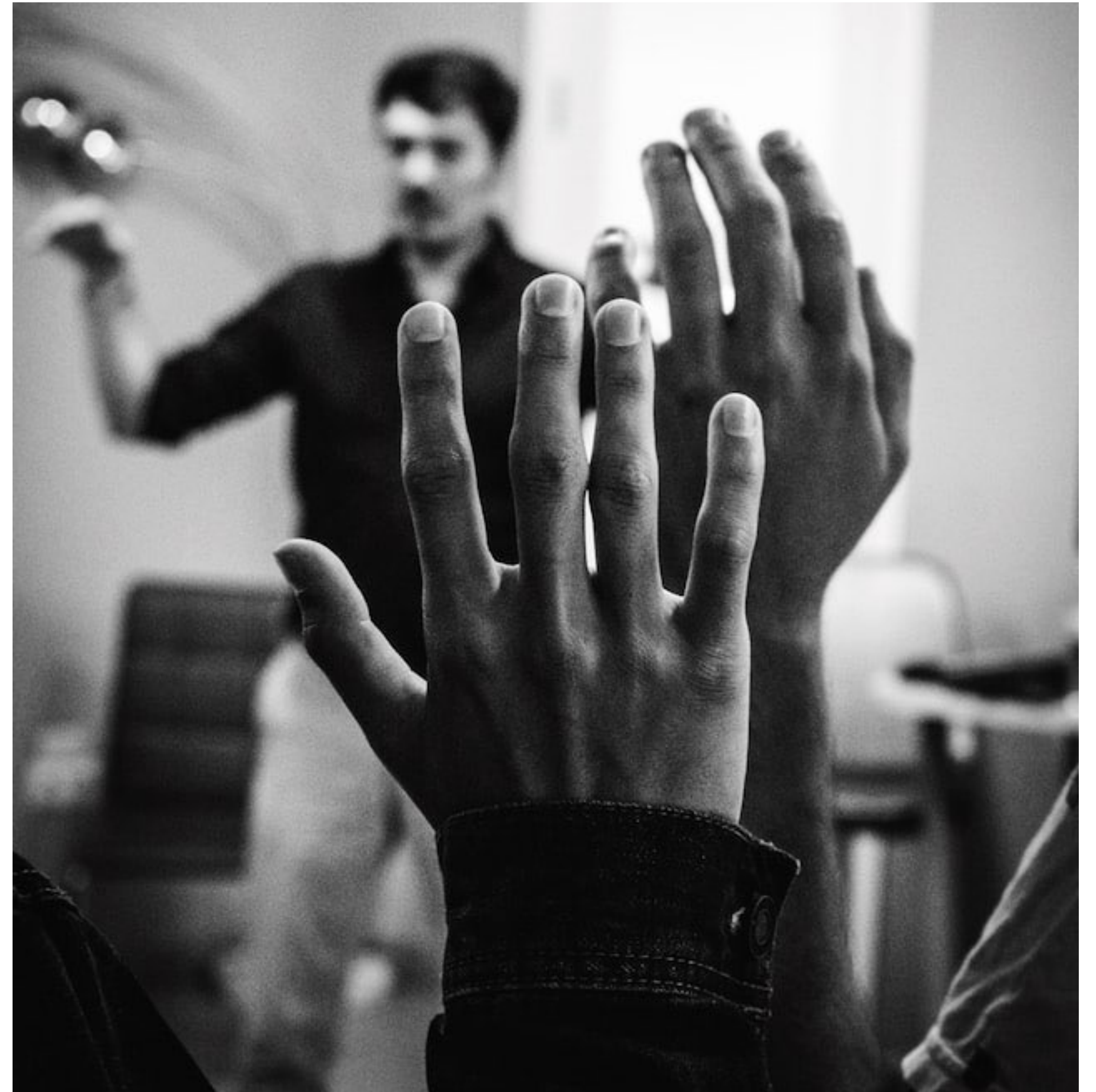
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Konstruktoren und Garbage Collection?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Konstrukturen und Garbage Collection



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

